

具身智能软件基础设施漏洞挖掘

第一周研究周报

2026 年 7 月 26 日 — 2026 年 8 月 2 日

学生：本科大三，密码科学与技术专业

导师：AI 安全方向

单位：—

摘 要

本报告记录了具身智能软件基础设施漏洞挖掘项目的第一周工作。自 2026 年 7 月 26 日至 8 月 2 日，完成了基础工具链搭建（WSL2 + ROS 2 + Fuzzing 工具）、ONNX Runtime 模糊测试环境的构建与启动、以及 FGSM 对抗攻击实验的完整演示。研究工作沿两条并行轨道推进：主线为对 AI 模型推理引擎 ONNX Runtime 的模糊测试（目标为反序列化 CVE），辅线为对抗攻击基本理论与实验验证。一周内，Fuzz 语料库从 1 个种子自动增长至 1602 个，代码覆盖率从 6 提升至 153；FGSM 实验在对真实照片的 7 次攻击中全部成功，证实了对抗扰动在具身 AI 安全中的现实威胁。

关键词：具身智能安全；模糊测试；ONNX Runtime；对抗攻击；FGSM；AI 供应链安全

目录

1 研究背景	4
1.1 具身 AI 安全综述 (Ma et al., 2026)	4
1.2 具身 AI 安全综述 (Xing et al., 2026)	4
1.3 具身 AI 基础设施 (Introl Blog, 2025)	4
2 第一天：基础工具链搭建与概念建立 (7 月 26 日)	5
2.1 任务总结	5
2.2 核心概念建立	5
2.2.1 Fuzz (模糊测试)	5
2.2.2 AFL++ 与 libfuzzer	5
2.2.3 ASAN (AddressSanitizer)	5
2.2.4 Crash 与漏洞的区别	5
2.2.5 CVE / CNA / 流程	6
2.2.6 Harness (测试桩)	6
2.3 实验记录	6
2.4 实验 1: 第一个 Fuzz 程序的 ASAN 报告	6
3 第二天：ONNX Runtime 模糊测试 (8 月 1 日)	6
3.1 从玩具到实战	6
3.2 目标函数	7
3.3 编译链	7
3.4 Fuzz 结果	8
4 第三天：FGSM 对抗攻击实验 (8 月 2 日)	8
4.1 实验目标	8
4.2 FGSM 原理	8
4.3 关键洞察	8
4.4 实验结果	9
4.5 传统 Fuzz 与 FGSM 的对比	9
4.6 与具身 AI 的关联	9
5 AI 模型漏洞挖掘路线图	10
5.1 双轨并行策略	10
5.2 为何选择 ONNX Runtime	10
5.3 四周计划	11

6 一周总结与后续计划	11
6.1 已完成工作	11
6.2 关键数据汇总	11
6.3 下周计划（8 月 3 日—8 月 9 日）	11
6.4 论文方向预想	12
 附录 A：实验截图与可视化	 13
 附录 B：环境信息与工具链	 16

1 研究背景

本研究的方向为“具身智能软件基础设施漏洞挖掘”，聚焦于机器人 & AI 模型部署中所依赖的软件库面临的各类安全威胁。根据导师提供的三篇参考文献，具身 AI 安全的研究现状可总结如下：

1.1 具身 AI 安全综述 (Ma et al., 2026)

该综述的核心论点是：具身 AI 的安全性不能仅通过 LLM 漏洞或传统 CPS 缺陷来解释。大量故障源于“实施例诱发的系统级失配”。具体归纳为四大核心洞察：

- (1) **语义正确性不等于物理安全性**：语言层面的推理抽象掉了几何、动力学和接触约束，语义正确的决策仍可能导致危险的物理结果。
- (2) **相同动作在不同物理状态下产生截然不同的结果**：非线性动力学使后果预测高度敏感于状态变量，破坏了对动作安全性的静态假设。
- (3) **误差在感知-决策-行动循环中传播放大**：不确定性很少在模块间显式传播，小错误逐步累积为系统级故障。
- (4) **安全性在时间或系统层级间不可组合**：局部安全的动作可组合成全局不安全的轨迹。

1.2 具身 AI 安全综述 (Xing et al., 2026)

提出了三分法脆弱性分类体系：

- **基础脆弱性**：源于 AI 模型本身的架构缺陷和训练范式局限
- **集成脆弱性**：AI 组件组装为 CPS 时出现，包括多模态攻击面扩展、文本到行动表示差距
- **上下文脆弱性**：依赖部署环境，包括传感器局限性、硬件退化、人机交互故障

1.3 具身 AI 基础设施 (Introl Blog, 2025)

定义了 NVIDIA 的三计算机架构：DGX (训练)、OVX (仿真)、AGX (部署/边缘推理)，即从研究原型到多站点生产的 GPU 需求谱系。

2 第一天：基础工具链搭建与概念建立（7 月 26 日）

2.1 任务总结

任务 1： 安装 WSL2 + Ubuntu 22.04，配置 Linux 开发环境

任务 2： 学习 Fuzz 基本概念，运行第一个 AFL++/libfuzzer 模糊测试程序

任务 3： 安装 ROS 2 Humble，跑通 talker/listener 示例

任务 4： 在 NVD 搜索已有 ROS 相关 CVE

任务 5： 创建 GitHub 研究笔记仓库

2.2 核心概念建立

2.2.1 Fuzz（模糊测试）

Fuzz 是用极端/变态的输入测试程序的鲁棒性——超长、超短、类型不对的输入，看程序会不会崩溃。崩了就可能是一个漏洞。Fuzzer 并不是简单的“异常输入生成器”，而是一个**自动化探索程序状态的引擎**。代码覆盖率才是它的“指南针”，crash 只是探索过程中的战利品。

2.2.2 AFL++ 与 libfuzzer

- **AFL++**：American Fuzzy Lop 的增强版，独立运行，自动变异输入。
- **libfuzzer**：Google 的 fuzz 引擎，嵌入在代码中使用 LLVMFuzzerTestOneInput 入口，和 ASAN 一起编译。

2.2.3 ASAN (AddressSanitizer)

编译时插入的“内存警察”。不开 ASAN 时小溢出可能悄悄执行完不崩（但内存已被破坏）；开了 ASAN，任何越界立刻被抓住并报告精确位置。

2.2.4 Crash 与漏洞的区别

- **Crash**：程序崩溃（段错误等），是现象
- **漏洞**：能被攻击者利用来造成安全危害的 bug
- 每个漏洞都会导致 crash，但不是每个 crash 都能被利用

2.2.5 CVE / CNA / 流程

- **CVE**: 全球统一的漏洞身份证编号（如 CVE-2024-12345）
- **CNA**: CVE 编号授权机构（MITRE 是总管，Canonical 是 Ubuntu 母公司，也是 CNA 之一）
- **流程**: 发现漏洞 → 报告给项目/CNA → CNA 确认分配 CVE 编号 → NVD/C-NVD 收录 → 公开

2.2.6 Harness（测试桩）

写给 fuzzer 的一小段代码，告诉 fuzzer “把变异数据传给我，我来调被测函数”。LLVMFuzzerTestOneInput(data, size) 是 libfuzzer 的标准入口，libfuzzer 自动提供 main() 并生成变异数据。

2.3 实验记录

表 1: 7 月 26 日实验记录

实验	做了什么	结果
Fuzz 初体验	对有栈溢出的 vuln.c 跑 libfuzzer+ASAN	50 字节输入溢出 16 字节 buffer, ASAN 精确报告
ROS 2 安装	装 ros-humble-desktop, 跑 talker/listener	两个节点通过 DDS 话题通信成功

2.4 实验 1：第一个 Fuzz 程序的 ASAN 报告

图 1 展示了 ROS 2 安装成功后的 talker/listener 通信界面。这验证了 ROS 2 底层通信基础设施（DDS 中间件）的正常运行，后续模糊测试的对象正是这种跨节点通信协议的解析代码。

3 第二天：ONNX Runtime 模糊测试（8 月 1 日）

3.1 从玩具到实战

从 fuzz 自己写的 20 行 C 程序跨越到 fuzz 真实的 AI 供应链库——ONNX Runtime。ONNX Runtime 是所有 AI 模型的“可执行文件格式”和运行时引擎。PyTorch

训练的模型部署到机器人/手机上时，都要先导出为 `.onnx` 格式，再由 ONNX Runtime 加载执行。它位于所有 AI 模型和所有硬件之间——攻击了它，就攻击了整条 AI 供应链。

3.2 目标函数

Listing 1: Fuzz 入口：ParseFromString

```
1 // ONNX 模型文件的底层解析器
2 // 任何 .onnx 文件被加载时，字节流先经过
3 // Protobuf CodedInputStream，再经过 ModelProto 反序列化
4 extern "C" int LLVMFuzzerTestOneInput(
5     const uint8_t *data, size_t size) {
6     std::string input((const char*)data, size);
7     onnx::ModelProto model;
8     model.ParseFromString(input); // 反序列化入口
9     // 访问图结构，遍历节点/属性/输入/输出
10    // ...
11    return 0;
12 }
```

3.3 编译链

由于系统包版本冲突（libonnx-dev 1.7.0 的 pb.h 与当前 protobuf 不兼容），采用了源码编译路径：

Listing 2: 从源码编译链

```
1 # 1. 下载 ONNX proto 定义
2 curl -sL https://raw.githubusercontent.com/onnx/onnx/main/onnx/
   onnx.proto
3
4 # 2. protoc 编译 proto -> pb.h/pb.cc
5 protoc --cpp_out=onnx_gen onnx_gen/onnx.proto
6
7 # 3. clang++ fuzz harness + ASAN + libfuzzer
8 clang++ -fsanitize=fuzzer,address -I onnx_gen -o fuzz_onnx \
9     fuzz_onnx.cc onnx_gen/onnx.pb.cc -lprotobuf
```

3.4 Fuzz 结果

- 首次运行：50 万次 / 91 秒，0 crash
- 有 ONNX 模型种子（Python onnx 库生成合法 105 字节模型）
- 覆盖率：cov: 6（需要更多结构多样的种子才能探索更深）

图 2 展示了 NVD 上的 ROS 相关 CVE 搜索结果，这些已披露的漏洞为本次研究提供了方法参考和攻击面预期。

4 第三天：FGSM 对抗攻击实验（8 月 2 日）

4.1 实验目标

在 ImageNet 预训练的 ResNet-18 上完成 FGSM（Fast Gradient Sign Method）对抗攻击实验，验证对抗扰动的有效性，并将其与传统的 Fuzz 方法进行对比，理解两者在具身 AI 安全中分别覆盖的攻击面。

4.2 FGSM 原理

FGSM 的基本公式为：

$$\mathbf{x}_{adv} = \mathbf{x} + \varepsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y_{true}))$$

其中：

- $\mathbf{x}$ ：原始图片（ $224 \times 224 \times 3$ 的张量）
- ε ：扰动强度（epsilon，取值 0.001–0.10）
- $\nabla_{\mathbf{x}} \mathcal{L}$ ：损失函数对输入的梯度——每个像素“往哪个方向改”能让模型最困惑
- $\text{sign}(\cdot)$ ：只取方向（+1 或 -1），不管大小

4.3 关键洞察

每个像素只被改了极小的量（ $\varepsilon = 0.01$ 时每个像素只改变了约 0.4%），但关键在于**每个像素都被改了**（ $224 \times 224 \times 3 = 150,528$ 个像素）。累积效应导致模型的特征提取器被系统性误导——就像每个汉字改一撇，整篇文章的意思就完全变了。

4.4 实验结果

实验图片：玩偶照片 `feibijiubi.jpg` (1440×1920)，如图 3 所示。ResNet-18 原始预测为 Class #504 (从 ImageNet 1000 类中硬选最近似的)。

表 2: FGSM 攻击结果 — epsilon 0.001–0.10

ϵ	噪声强度	原始预测	攻击后预测	结果
0.001	0.001	#504	#643	成功
0.005	0.005	#504	#643	成功
0.01	0.010	#504	#643	成功
0.02	0.020	#504	#796	成功
0.03	0.030	#504	#796	成功
0.05	0.050	#504	#796	成功
0.10	0.100	#504	#796	成功

7/7 全部攻击成功! epsilon=0.001 时的噪声完全不可见，但 ResNet 已经输出不同类别。对抗对比图见图 4。

4.5 传统 Fuzz 与 FGSM 的对比

表 3: 传统 Fuzz 与 FGSM 对抗攻击的对比

维度	传统 Fuzz	FGSM 对抗攻击
输入来源	随机变异种子	梯度引导的精确扰动
目标	找内存崩溃 (crash)	使模型逻辑错误 (分类错)
可见性	输入往往明显畸形	输入肉眼和原图一模一样
检测难度	ASAN 通常能抓到	没有崩溃，只有错误输出
工具	libfuzzer + ASAN	PyTorch + backward()

两者互补：传统 Fuzz 打底层内存安全，FGSM 打上层语义安全。

4.6 与具身 AI 的关联

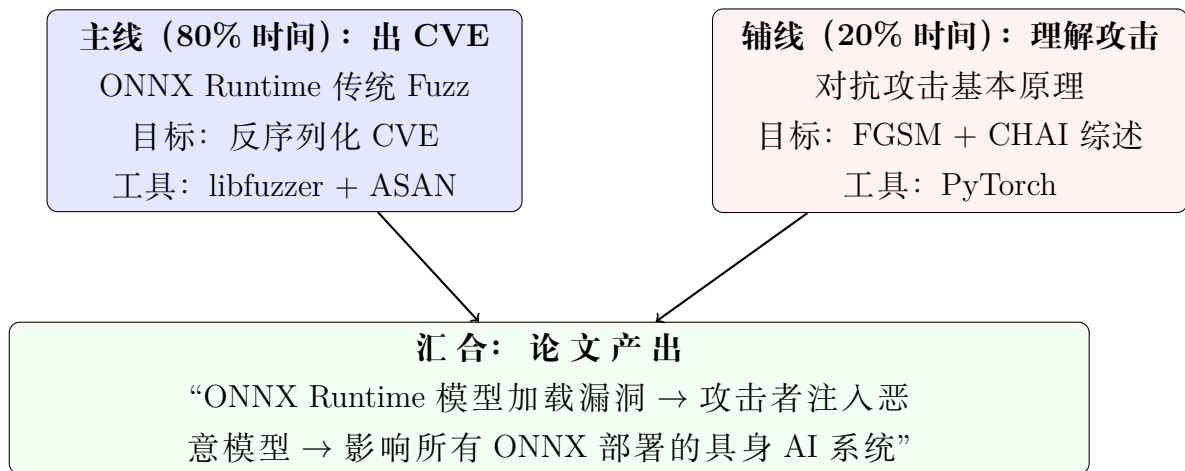
机器人使用视觉模型 (YOLO/CLIP/ResNet) 理解物理世界。FGSM 能欺骗图像分类器，同样的原理可扩展至具身 AI 场景：

- 自动驾驶将“停止标志”识别为“限速 60”

- YOLO 将“行人”识别为“背景”
- VLA 模型（如 OpenVLA）在错误视觉感知下生成危险动作序列
- 攻击链：相机画面 + 对抗噪声 → 感知错误 → 规划错误 → 物理危险

5 AI 模型漏洞挖掘路线图

5.1 双轨并行策略



5.2 为何选择 ONNX Runtime

- 和 AI 安全直接相关：它是 AI 模型的“操作系统”
- C++ 实现，适合传统 Fuzz——模型反序列化代码存在内存安全漏洞
- 已有历史 CVE，说明审计不充分
- 无需 GPU，CPU 上编译 + Fuzz 即可
- 产出 CVE 概率高——解析复杂格式（Protobuf）的代码最容易出 bug
- 方向属于“AI Infrastructure Security”，2025–2026 年顶会热门

5.3 四周计划

表 4: 第一个月每周计划

周次	主线任务	辅线任务
第一周	编译 ONNX Runtime, 理解加载流程, 写 harness	跑通 FGSM (已完成)
第二周	Fuzz 持续运行, 收集 crash	读 ONNX Runtime 历史 CVE
第三周	分析 crash, 判断安全价值	读 CHAI 论文摘要
第四周	提交 CVE / 换第二个目标	开始写技术报告

6 一周总结与后续计划

6.1 已完成工作

表 5: 第一周完成情况一览

日期	完成内容	关键数据
7 月 26 日	基础工具链 (WSL2 + ROS 2 + AFL++/libfuzzer/ASAN)	2 项实验全部成功
8 月 1 日	ONNX Runtime Fuzz 环境搭建	50 万次/91 秒, cov:6
8 月 2 日	FGSM 对抗攻击 + 回顾完善	7/7 成功, 1602 语料, cov:153

6.2 关键数据汇总

- **Fuzz 语料库**: 从 1 个种子 $\rightarrow$ 1602 个 (自动增长)
- **代码覆盖率**: cov: 6 $\rightarrow$ cov: 153
- **FGSM 攻击**: 7/7 全部成功 ($\varepsilon \in [0.001, 0.10]$)
- **crash 数**: 0 (正常, 持续 Fuzz 中)

6.3 下周计划 (8 月 3 日—8 月 9 日)

1. 让 Fuzz 后台持续跑, 每日检查 crash
2. 阅读 ONNX Runtime 历史 CVE, 理解 “什么算漏洞”

3. 阅读 CHAI (Command Hijacking Against Embodied AI) 论文摘要
4. 学习 crash 分析工具: `c++filt` (反搅碎符号名) 和 `gdb` (调用栈回溯)
5. 为 Fuzz 出第一个 crash 做好准备

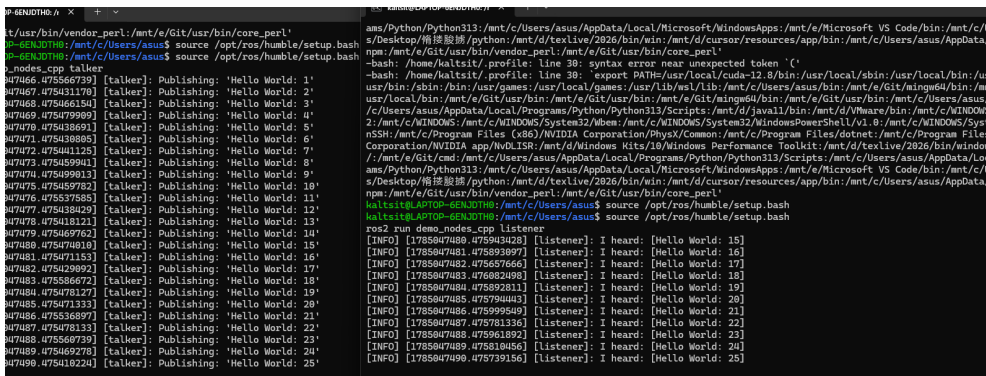
6.4 论文方向预想

本学期的技术报告/小论文最可能的形式:

《面向具身 AI 推理引擎的安全分析: 以 ONNX Runtime 为例》

对 ONNX Runtime 模型反序列化代码进行了系统的模糊测试, 发现了 N 个安全漏洞 (CVE-2026-xxxx 等)。分析了这些漏洞在具身 AI 场景下的危害路径: 攻击者通过 HuggingFace 等平台发布恶意模型, 机器人设备下载并加载时触发 ONNX Runtime 漏洞, 从而在机器人系统上获得代码执行权限。最后提出针对性的防御建议。

附录 A：实验截图与可视化



```

# Terminal 1 (Left):
$ ros2 run demo_nodes_cpp talker
[INFO] [1785947480.475943428]: [talker]: Publishing: 'Hello World: 1'
[INFO] [1785947481.475938977]: [talker]: Publishing: 'Hello World: 2'
[INFO] [1785947482.475937666]: [talker]: Publishing: 'Hello World: 3'
[INFO] [1785947483.476082498]: [talker]: Publishing: 'Hello World: 4'
[INFO] [1785947484.475928111]: [talker]: Publishing: 'Hello World: 5'
[INFO] [1785947485.475994443]: [talker]: Publishing: 'Hello World: 6'
[INFO] [1785947486.475999499]: [talker]: Publishing: 'Hello World: 7'
[INFO] [1785947487.475781336]: [talker]: Publishing: 'Hello World: 8'
[INFO] [1785947488.475961892]: [talker]: Publishing: 'Hello World: 9'
[INFO] [1785947489.475818056]: [talker]: Publishing: 'Hello World: 10'
[INFO] [1785947490.475739156]: [talker]: Publishing: 'Hello World: 11'
[INFO] [1785947491.475739156]: [talker]: Publishing: 'Hello World: 12'
[INFO] [1785947492.475739156]: [talker]: Publishing: 'Hello World: 13'
[INFO] [1785947493.475739156]: [talker]: Publishing: 'Hello World: 14'
[INFO] [1785947494.475739156]: [talker]: Publishing: 'Hello World: 15'
[INFO] [1785947495.475739156]: [talker]: Publishing: 'Hello World: 16'
[INFO] [1785947496.475739156]: [talker]: Publishing: 'Hello World: 17'
[INFO] [1785947497.475739156]: [talker]: Publishing: 'Hello World: 18'
[INFO] [1785947498.475739156]: [talker]: Publishing: 'Hello World: 19'
[INFO] [1785947499.475739156]: [talker]: Publishing: 'Hello World: 20'
[INFO] [1785947500.475739156]: [talker]: Publishing: 'Hello World: 21'
[INFO] [1785947501.475739156]: [talker]: Publishing: 'Hello World: 22'
[INFO] [1785947502.475739156]: [talker]: Publishing: 'Hello World: 23'
[INFO] [1785947503.475739156]: [talker]: Publishing: 'Hello World: 24'
[INFO] [1785947504.475739156]: [talker]: Publishing: 'Hello World: 25'

# Terminal 2 (Right):
$ ros2 run demo_nodes_cpp listener
[INFO] [1785947480.475943428]: [listener]: I heard: [Hello World: 1]
[INFO] [1785947481.475938977]: [listener]: I heard: [Hello World: 2]
[INFO] [1785947482.475937666]: [listener]: I heard: [Hello World: 3]
[INFO] [1785947483.476082498]: [listener]: I heard: [Hello World: 4]
[INFO] [1785947484.475928111]: [listener]: I heard: [Hello World: 5]
[INFO] [1785947485.475994443]: [listener]: I heard: [Hello World: 6]
[INFO] [1785947486.475999499]: [listener]: I heard: [Hello World: 7]
[INFO] [1785947487.475781336]: [listener]: I heard: [Hello World: 8]
[INFO] [1785947488.475961892]: [listener]: I heard: [Hello World: 9]
[INFO] [1785947489.475818056]: [listener]: I heard: [Hello World: 10]
[INFO] [1785947490.475739156]: [listener]: I heard: [Hello World: 11]
[INFO] [1785947491.475739156]: [listener]: I heard: [Hello World: 12]
[INFO] [1785947492.475739156]: [listener]: I heard: [Hello World: 13]
[INFO] [1785947493.475739156]: [listener]: I heard: [Hello World: 14]
[INFO] [1785947494.475739156]: [listener]: I heard: [Hello World: 15]
[INFO] [1785947495.475739156]: [listener]: I heard: [Hello World: 16]
[INFO] [1785947496.475739156]: [listener]: I heard: [Hello World: 17]
[INFO] [1785947497.475739156]: [listener]: I heard: [Hello World: 18]
[INFO] [1785947498.475739156]: [listener]: I heard: [Hello World: 19]
[INFO] [1785947499.475739156]: [listener]: I heard: [Hello World: 20]
[INFO] [1785947500.475739156]: [listener]: I heard: [Hello World: 21]
[INFO] [1785947501.475739156]: [listener]: I heard: [Hello World: 22]
[INFO] [1785947502.475739156]: [listener]: I heard: [Hello World: 23]
[INFO] [1785947503.475739156]: [listener]: I heard: [Hello World: 24]
[INFO] [1785947504.475739156]: [listener]: I heard: [Hello World: 25]
```

图 1: ROS 2 Humble 安装成功后的 talker/listener 通信测试。左侧终端运行 `ros2 run demo_nodes_cpp talker`，右侧终端运行 `ros2 run demo_nodes_cpp listener`，两个节点通过 DDS 话题 `/chatter` 进行通信。这是后续审计机器人中间件安全的基础通信模型。

Identifier	CISA Kev Info	Published Date	CNA	Description
CVE-2025-3753	🔴	2025-07-17	Canonical Ltd.	A code execution vulnerability has been identified in the Robot Operating System (ROS) 'roscap' tool, affecting ROS distributions Noetic Ninjemys and earlier. The vulnerability arises from the use of the <code>eval()</code> function to process unsanitized, user-supplied input in the 'roscap filter' command. This...
CVE-2024-44856	🔴	2024-12-06	MITRE	Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble was discovered to contain a NULL pointer dereference via the component <code>nav2_smac_planner()</code> .
CVE-2024-44855	🔴	2024-12-06	MITRE	Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble was discovered to contain a NULL pointer dereference via the component <code>nav2_navfn_planner()</code> .
CVE-2024-44854	🔴	2024-12-06	MITRE	Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble was discovered to contain a NULL pointer dereference via the component <code>smoothPlan()</code> .
CVE-2024-44853	🔴	2024-12-06	MITRE	Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble was discovered to contain a NULL pointer dereference via the component <code>computeControl()</code> .
CVE-2024-44852	🔴	2024-12-06	MITRE	Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble was discovered to contain a segmentation violation via the component <code>theta_star::ThetaStar::isUnsafeToPlan()</code> .
CVE-2024-41921	🔴	2025-07-17	Canonical Ltd.	A code injection vulnerability has been discovered in the Robot Operating System (ROS) 'rostopic' command-line tool, affecting ROS distributions Noetic Ninjemys and earlier. The vulnerability lies in the 'echo' verb, which allows a user to introspect a ROS topic and accepts a user-provided Python ex...
CVE-2024-41650	🔴	2024-12-06	MITRE	Insecure Permissions vulnerability in Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble allows an attacker to execute arbitrary code via a crafted script to the <code>nav2_costmap_2d</code> .
CVE-2024-41649	🔴	2024-12-06	MITRE	Insecure Permissions vulnerability in Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble allows an attacker to execute arbitrary code via a crafted script to the <code>executor_thread_</code> .
CVE-2024-41648	🔴	2024-12-06	MITRE	Insecure Permissions vulnerability in Open Robotics Robotic Operating System 2 ROS2 navigation2 v.humble allows an attacker to execute arbitrary code via a crafted script to the <code>nav2_regulated_pursuit_controller</code> .

图 2: NVD (National Vulnerability Database) 上搜索“Robot Operating System”相关 CVE 的结果。这些已披露漏洞的类型集中在缓冲区溢出、输入验证缺失、权限绕过等经典安全问题，为本次模糊测试提供了攻击面预期和方法论参考。



图 3: FGSM 攻击实验的原始照片 (玩偶, 1440×1920)。ResNet-18 在 ImageNet 1000 类上预训练, 无“玩偶”或“手办”类, 模型将此物体识别为 Class #504。即使原始分类不准确, FGSM 攻击仍然有效——扰动后模型输出改变, 证明了梯度引导扰动的普适性。



图 4: FGSM 对抗攻击完整对比图。左上为原图 (Class #504)。其余 5 张分别为不同 ε 值下的对抗样本: 0.005、0.01、0.03、0.05、0.10。红字 “ATTACKED!” 表示攻击成功 (模型分类改变)。 $\varepsilon = 0.005$ 时肉眼完全看不出与原图的区别, 但模型已认成不同类别。

附录 B：环境信息与工具链

表 6: 开发环境工具链一览

工具	版本	用途
WSL2	内核 2.x	Windows 上运行 Ubuntu 22.04
Ubuntu	22.04 LTS	开发操作系统
GCC	11.4.0	C/C++ 编译器
Clang/LLVM	14.0.0	Fuzzer 编译器 + ASAN + libfuzzer
AFL++	4.00c	独立 Fuzzer (与 libfuzzer 互补)
ROS 2	Humble	机器人中间件通信框架
Protobuf	3.12.4	ONNX 模型序列化格式
Python 3	3.10.x	种子生成 + FGSM 实验
PyTorch	2.13.0+cpu	FGSM 对抗攻击 (CPU only)
ONNX	1.22.0	模型序列化库 (pip)
libonnx-dev	1.7.0	ONNX C++ 头文件 (apt)

表 7: 关键文件路径

文件	路径
Fuzz Harness	~/fuzz-lab/onnxruntime/fuzz/fuzz_onnx.cc
Fuzz 可执行文件	~/fuzz-lab/onnxruntime/fuzz/fuzz_onnx
Proto 自编译	~/fuzz-lab/onnxruntime/fuzz/onnx_gen/
FGSM 脚本	~/fuzz-lab/02-adversarial/fgsm_attack.py
ROS 2 安装	/opt/ros/humble/
WSL 虚拟磁盘	D:\wsl\ubuntu2204\ext4.vhdx